Healthcare Appointment & Follow-up Manager

Final Project Requirements & Analysis Reference

Purpose of this document: This is the master reference for the project. The Original Requirements section represents what the project explicitly asks for. The Requirement Analysis section clarifies what those requirements mean for implementation without unnecessarily adding new mandatory features.

1. Project Title

Healthcare Appointment & Follow-up Manager

2. Project Objective

A clinic needs more than a basic appointment booking form.

Patients should be able to:

- Share their symptoms before an appointment.
- Book appointments.
- Receive appointment reminders.
- Receive timely confirmations.

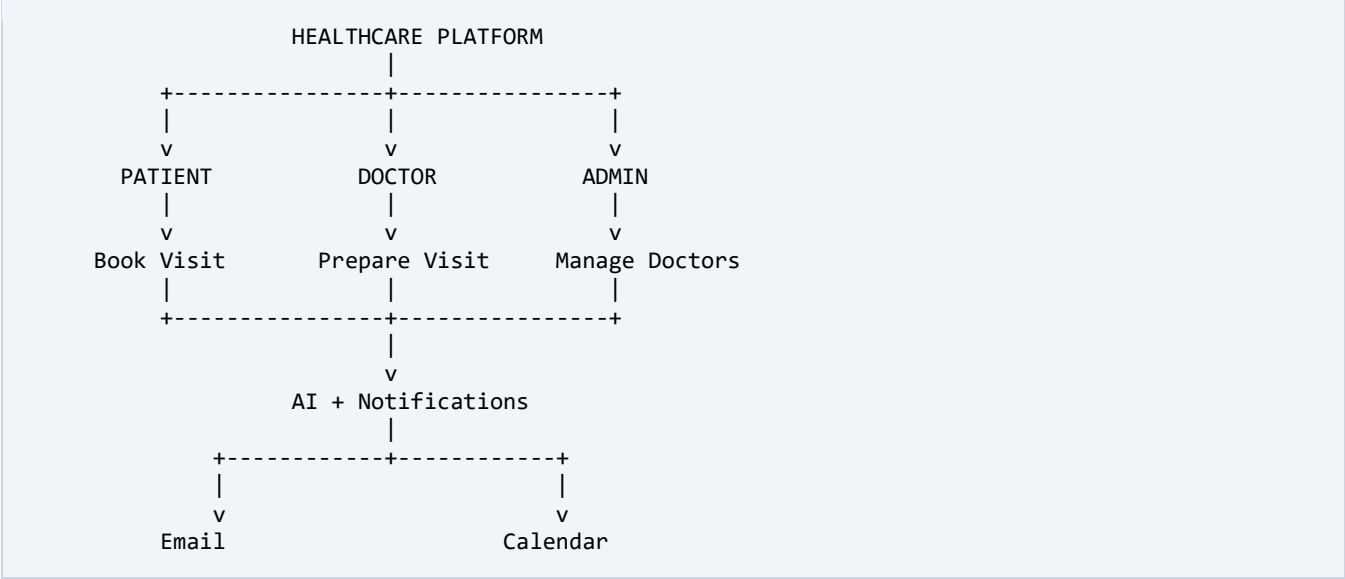
Doctors should be able to:

- View a concise AI-generated summary of the patient's symptoms before the visit.
- Record post-visit notes and prescriptions.

The system should also:

- Generate a patient-friendly post-visit summary.
- Send notifications through email.
- Create and synchronize Google Calendar events.
- Provide separate portals for patients, doctors, and administrators.

Core Objective



3. Scope of Work

3.1 Inputs

The system accepts:

- Patient Symptoms
- Appointment Details
- Doctor Information
- Working Hours
- Slot Duration
- Leave Information
- Doctor Post-Visit Notes
- Prescription Information

The first two are explicitly identified as the primary project inputs; the others are required operational data.

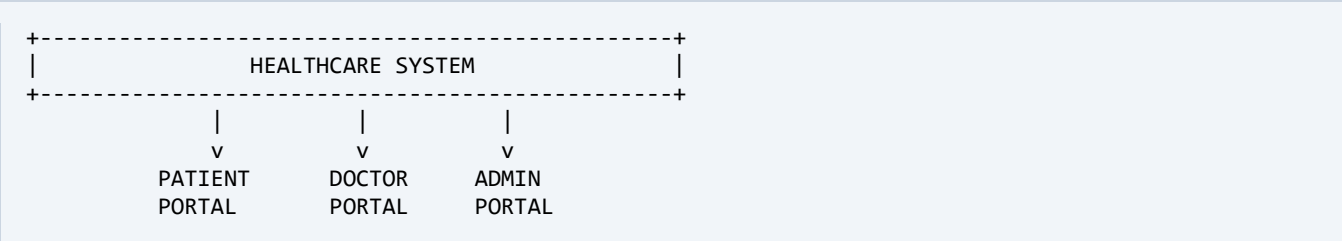
3.2 Outputs

The system produces:

- Booked Appointment
- AI Pre-Visit Symptom Summary
- Patient-Friendly Post-Visit Summary
- Medication Reminders
- Email Notifications
- Google Calendar Events

4. Users / Roles

The system has three roles:



Patient

Uses the system primarily for:

- Registration
- Login
- Doctor search
- Appointment booking
- Symptom submission
- Appointment management
- Viewing summaries
- Receiving reminders

Doctor

Uses the system primarily for:

- Login
- Viewing appointments
- Reviewing patient symptoms
- Reviewing AI pre-visit summary
- Entering post-visit notes
- Entering prescription
- Viewing/receiving relevant notifications

Admin

Uses the system primarily for:

- Creating doctors
- Managing doctors
- Managing specialisations
- Managing working hours
- Configuring slot duration
- Managing leave days

5. Functional Requirements

ID	Requirement	Description
FR-01	Patient Registration	The patient must be able to create an account.
FR-02	Patient Login	The patient must be able to log into the system.
FR-03	Doctor Login	Doctors must have authenticated access to their doctor portal.
FR-04	Admin Access	Administrators must have authenticated access to the admin portal.
FR-05	Role-Based Access	The system must distinguish between PATIENT, DOCTOR, and ADMIN and provide role-appropriate functionality.

6. Doctor Management

The admin must be able to create and manage doctor profiles.

A doctor profile must support:

```
Doctor
 |
 +-- Specialisation
 |
 +-- Working Hours
 |
 +-- Slot Duration
 |
 +-- Leave Days
```

7. Doctor Search

Patients must be able to:

- Search for doctors.
- Search/filter doctors by specialisation.

Example:

```
Patient
   |
   v
Search: "Cardiologist"
   |
   v
Available Cardiologists
   |
   v
Select Doctor
```

8. Doctor Availability

The system must determine available appointment slots using the doctor's:

- Working Hours
- Slot Duration
- Leave Days
- Existing Appointments

Example:

```
Working Hours: 10:00 AM - 12:00 PM
Slot Duration: 30 minutes

Generated Slots:
10:00 - 10:30
10:30 - 11:00
11:00 - 11:30
11:30 - 12:00
```

Unavailable slots should not be bookable.

9. Appointment Booking

The patient must be able to:

```
Search Doctor
     |
     v
Select Doctor
     |
     v
Select Date
     |
     v
View Available Slots
     |
     v
Select Slot
     |
     v
Enter Symptoms
     |
     v
Confirm Appointment
```

The final result is a booked appointment.

10. Slot Hold Mechanism

A slot hold mechanism is explicitly mentioned in the required system-design write-up.

The intended concept is:

```
AVAILABLE
    |
    v
  HELD
    |
    +---- Confirmed ----> BOOKED
    |
    +---- Expired ------> AVAILABLE
```

A temporary hold prevents a slot from being unnecessarily selected by multiple users during the booking process.

The exact implementation is a design decision.

11. Double-Booking Prevention

The system must prevent two users from successfully booking the same doctor's slot.

It must also safely handle simultaneous booking attempts.

Example:

```
Patient A --------\
                   \
                    > Doctor X
                   /  10:00 AM
Patient B --------/
```

```
Expected result:

Patient A -> SUCCESS
Patient B -> SLOT UNAVAILABLE
```

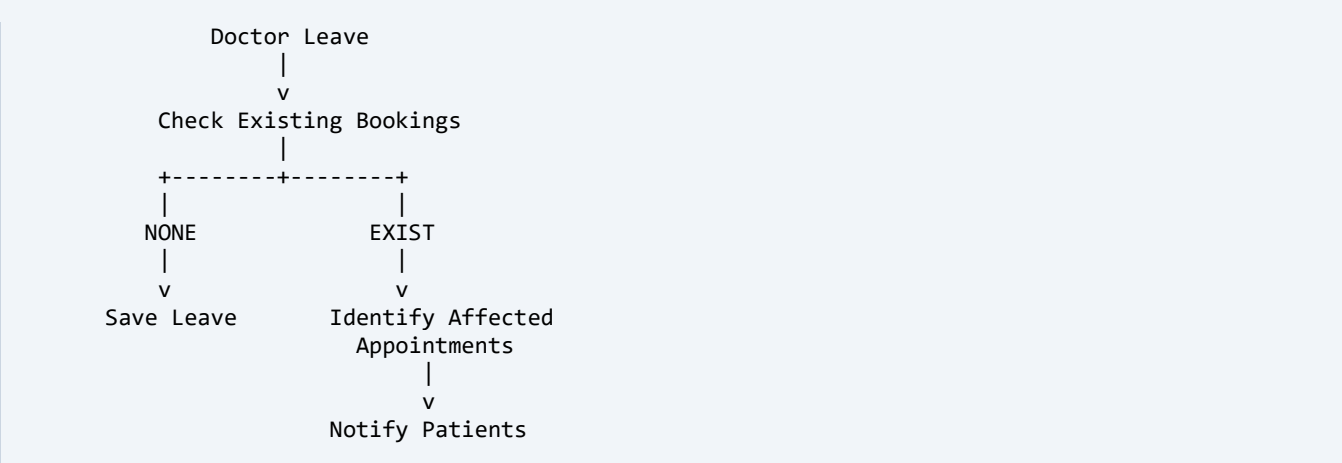
This is one of the most important engineering requirements of the project.

The exact implementation—database constraints, transactions, locking, slot holds, etc.—is part of the system design.

12. Doctor Leave Management

The admin can mark a doctor as being on leave.

When leave is added:

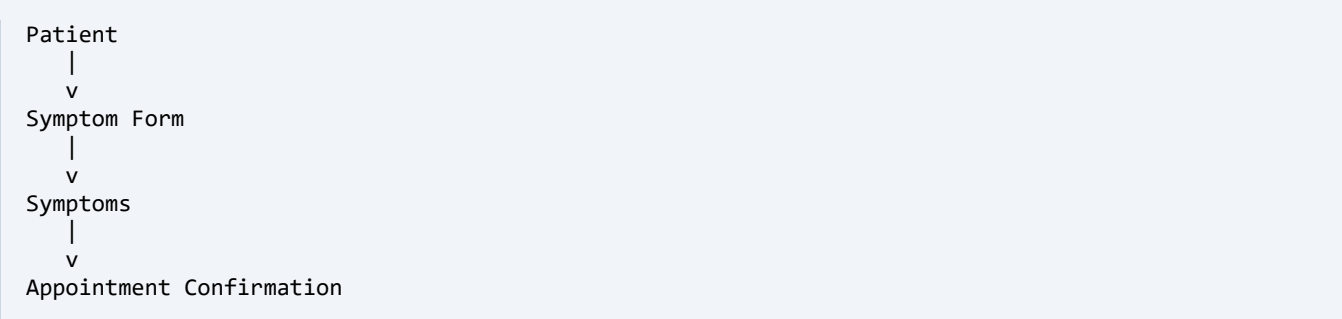


Patients affected by a doctor's leave must be notified.

The system should also prevent new appointments from being booked during the doctor's unavailable period.

13. Patient Symptom Form

Before confirming an appointment, the patient must provide symptoms.



The symptoms are used for the pre-visit AI summary.

14. Pre-Visit AI Summary

The LLM must generate a pre-visit summary for the doctor.

The required output consists of:

- Urgency Level
- Chief Complaint
- Three Suggested Questions for Doctor

Urgency must be: Low / Medium / High

> ***PROMPT:***
>
> `Analyse these symptoms and return: urgency level (Low / Medium / High), chief complaint, and three suggested questions for the doctor. Symptoms: <symptoms>`

The generated output must be stored in the database.

15. Doctor Consultation

After the appointment/consultation, the doctor submits:

- Post-Visit Notes
- Prescription

These become the basis for the post-visit summary.

16. Post-Visit AI Summary

The LLM must generate a patient-friendly summary from the clinical notes.

The summary should contain:

- Patient-Friendly Explanation
- Medication Schedule
- Follow-Up Steps

> ***PROMPT:***
>
> `Convert these clinical notes into a patient-friendly summary with medication schedule and follow-up steps: <notes>`

The generated output must be stored in the database.

17. Prescription & Medication Reminders

The system must use prescription frequency to generate medication reminders.

Conceptually:

```
Doctor Prescription
       |
       v
Medication Frequency
```

```
       |
       v
Reminder Schedule
       |
       v
Background Job
       |
       v
Medication Reminder
```

The project explicitly requires background processing for medication reminders.

18. Email Notifications

The system must send email notifications to both Patient and Doctor.

Required notification types:

- Booking Confirmation
- Appointment Reminder
- Cancellation

Doctor-leave-related notifications are also required for affected patients.

Possible email services include: SendGrid, Mailgun, Nodemailer, or a similar service.

The choice of provider is an implementation decision.

19. Email Retry

Email failures must be handled through a background job.

Conceptually:

```
Notification
     |
     v
Email Worker
     |
  +--+--+
  |     |
SUCCESS FAIL
  |     |
  v     v
 SENT   RETRY
          |
          v
       SUCCESS
          |
          v
       SENT
```

The exact retry policy is a design decision.

20. Google Calendar Integration

The system must integrate with Google Calendar using the Google Calendar API + OAuth 2.0.

20.1 Appointment Booking

When an appointment is booked:

```
Appointment
     |
     v
Google Calendar
     |
     +----> Patient Event
     |
     +----> Doctor Event
```

20.2 Rescheduling

When an appointment is rescheduled:

```
Appointment Rescheduled
          |
          v
Calendar Event Updated
```

20.3 Cancellation

When an appointment is cancelled:

```
Appointment Cancelled
          |
          v
Calendar Event Deleted
```

21. Appointment Lifecycle

The project requires booking, reminders, cancellation and rescheduling behavior.

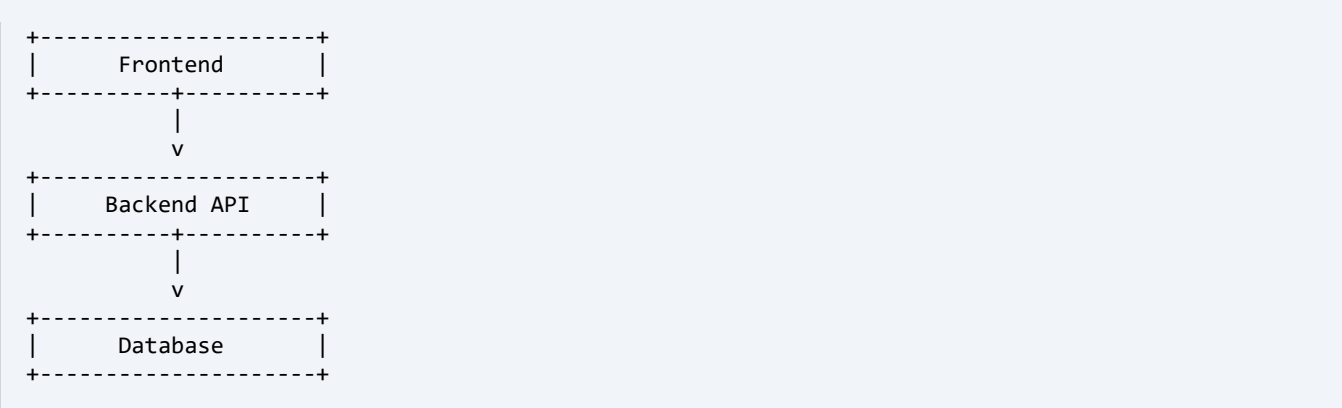
A conceptual lifecycle is:

```
AVAILABLE
    |
    v
  HELD
    |
    v
  BOOKED
    |
    +-------> CANCELLED
    |
    +-------> RESCHEDULED
    |
    v
 COMPLETED
```

The exact database status model is an implementation decision.

22. Technical Architecture Requirements

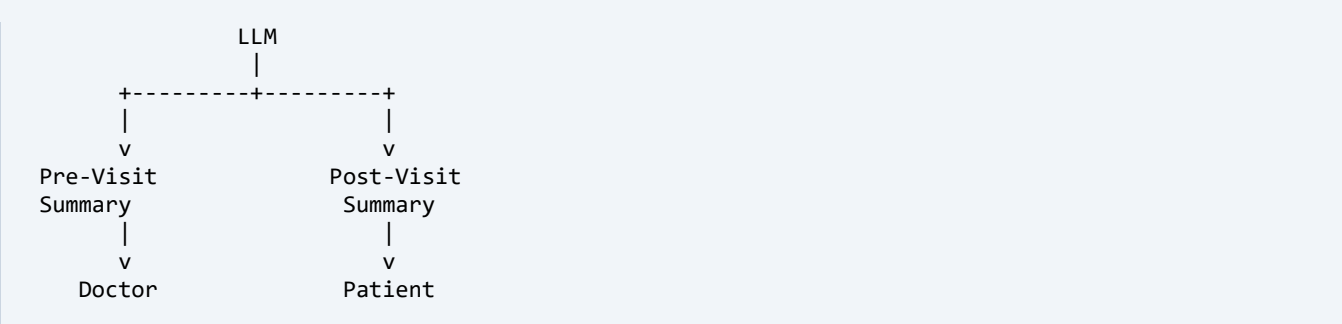
The project requires three primary application layers:



The application must support role-based authentication for: Patient, Doctor, Admin.

23. LLM Integration Requirements

There are two required LLM use cases:



Both generated outputs must be stored in the database.

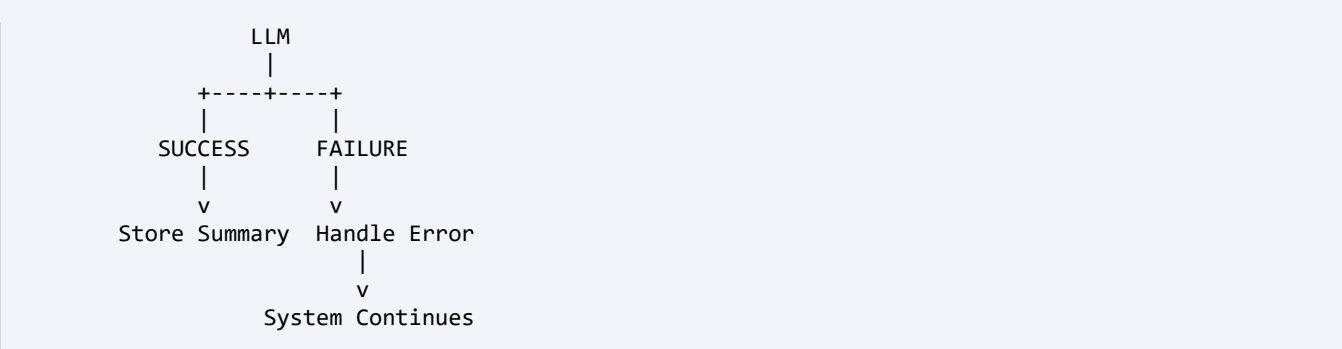
24. LLM Failure Handling

This is an explicit requirement.

LLM failures must be handled gracefully and must not break the system.

Therefore, the architecture should not make the entire healthcare application dependent on successful LLM execution.

Conceptually:



The precise fallback mechanism is a design decision.

25. Background Jobs

The project explicitly requires background jobs for:

- Medication Reminders
- Email Retries

Other background processing may be introduced if useful, but these two are the explicitly required uses.

26. Database Requirements

The database must store the information required for:

- Users
- Patients
- Doctors
- Doctor Availability
- Working Hours
- Slot Duration
- Leave Days
- Appointments
- Patient Symptoms
- Pre-Visit AI Summary
- Post-Visit Notes
- Prescription
- Post-Visit AI Summary
- Medication Reminder Information
- Notifications
- Calendar Integration Information

The exact database schema is an implementation decision.

27. Data Flow — Before Appointment

```
Patient
   |
   v
Search Doctor
   |
   v
Select Slot
   |
   v
Enter Symptoms
   |
   v
Appointment Booking
   |
   +------------------+
```

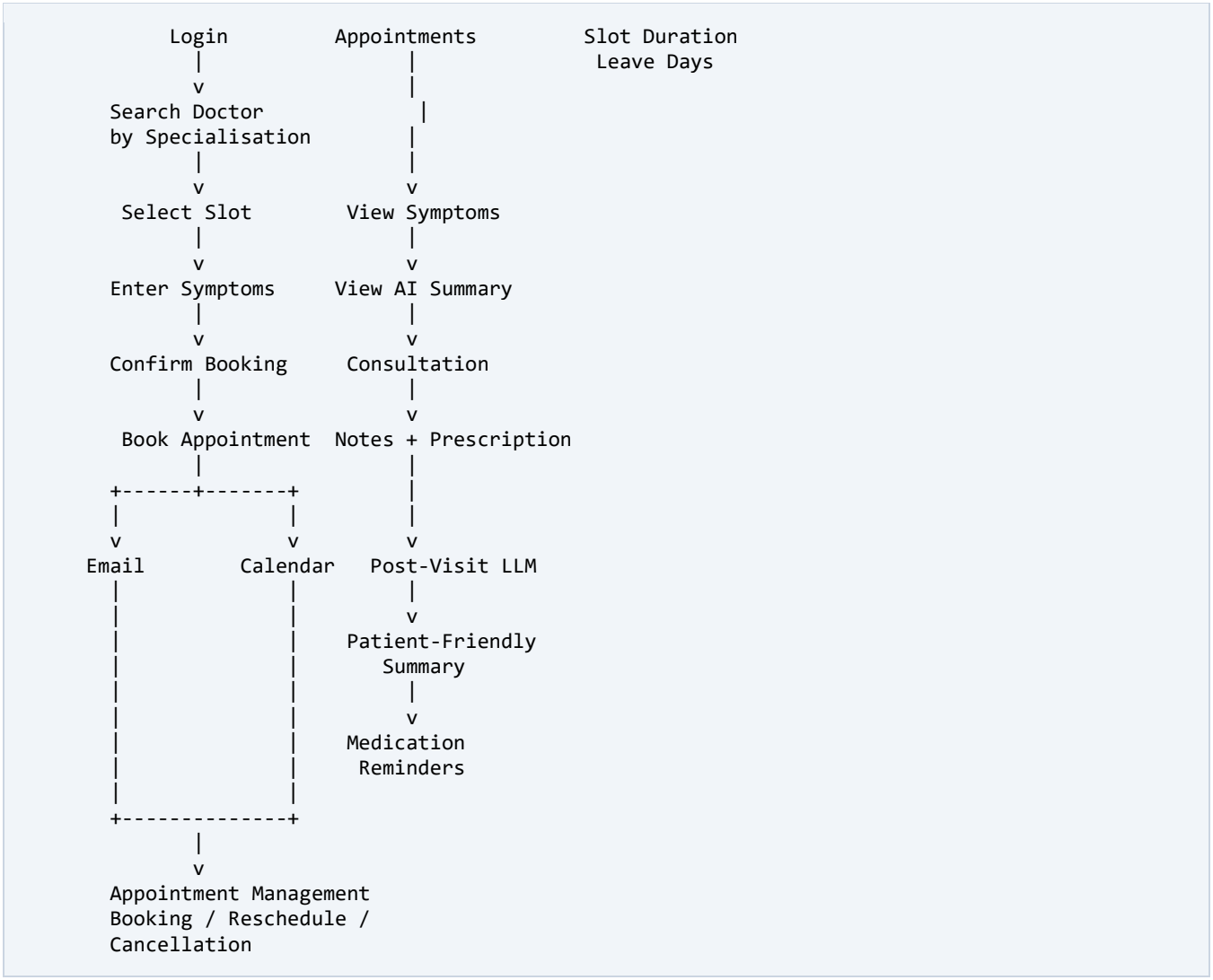
```
   |                  |
   v                  v
   DB                 LLM
   |                  |
   |                  v
   |              AI Summary
   |                  |
   |                  v
   |                Doctor
   |
   +-------> Email
   |
   +-------> Google Calendar
```

28. Data Flow — During / After Appointment

```
Doctor
   |
   v
View Appointment
   |
   v
View Patient Symptoms
   |
   v
View AI Pre-Visit Summary
   |
   v
Consultation
   |
   v
Enter Notes + Prescription
   |
   v
LLM
   |
   v
Patient-Friendly Summary
   |
   +-----------> Patient
   |
   v
Medication Reminder Schedule
```

29. Complete Functional Flow

```
                        HEALTHCARE SYSTEM
                                |
            +-----------------+-----------------+
            |                 |                 |
            v                 v                 v
         PATIENT           DOCTOR             ADMIN
            |                 |                 |
            v                 v                 v
        Register          Login            Manage Doctors
            |                 |             Specialisation
            v                 v             Working Hours
```



30. Required Deliverables

Deliverable 1 — Complete Source Code

A ZIP file containing the complete source code.

Deliverable 2 — README

The README must include:

- Setup Guide
- .env.example
- API Documentation
- Database Schema
- LLM Prompts
- Google Calendar Setup Steps

Deliverable 3 — Hosted Application

A hosted application URL. Possible free hosting services mentioned in the assignment include Vercel, Render, Railway, or another suitable free hosting platform.

Deliverable 4 — System Design Write-Up

Maximum: 800 words. The write-up must cover:

- 1. Double-booking prevention
- 2. Doctor leave conflict handling
- 3. Slot hold mechanism
- 4. Notification failure handling

31. Evaluation Criteria

The evaluator will focus on:

31.1 Slot Conflict Handling

- Slot Availability
- Double Booking
- Simultaneous Booking
- Slot Conflicts

31.2 Leave Management

- Doctor Leave
- Existing Bookings
- Affected Patients
- Notifications

31.3 Notification Reliability

- Email Sending
- Email Failure
- Email Retry

31.4 LLM Quality

- Pre-Visit Prompt
- Post-Visit Prompt
- Generated Output Quality

31.5 LLM Failure Handling

The system must continue functioning when the LLM fails.

31.6 Database Schema

- Database Design

- Relationships
- Appointment Data
- LLM Output Storage

31.7 API Design

- API Structure
- Endpoints
- Request/Response Design
- Code Organization

31.8 Code Structure

The project should have sensible separation between:

- Frontend
- Backend
- Database
- Services
- Background Jobs
- Integrations

31.9 Email Integration

Email functionality will be evaluated.

31.10 Google Calendar Integration

- OAuth 2.0
- Create Event
- Update Event
- Delete Event

31.11 Documentation

- README
- Setup Instructions
- .env.example
- API Documentation
- Database Schema
- LLM Prompts
- Google Calendar Setup

32. Core Modules

Based strictly on the requirements, the project can be divided into these logical modules:

```
+-------------------------------------------------------+
|                  APPLICATION MODULES                  |
+-------------------------------------------------------+
|                                                       |
|  1. Authentication & Role Management                  |
|  2. Patient Management                                |
|  3. Doctor Management                                 |
|  4. Admin Management                                  |
|  5. Doctor Availability / Slot Management             |
|  6. Appointment Management                            |
|  7. Doctor Leave Management                           |
|  8. Symptom Management                                |
|  9. Pre-Visit LLM Service                             |
| 10. Consultation / Post-Visit Management              |
| 11. Prescription & Medication Reminders               |
| 12. Email Notification Service                        |
| 13. Google Calendar Integration                       |
| 14. Background Job Processing                         |
|                                                       |
+-------------------------------------------------------+
```

These are logical modules, not necessarily separate microservices.

33. Core Domain Objects

The requirements naturally lead to objects/entities such as:

- User
- Patient
- Doctor
- Admin
- WorkingHour
- DoctorLeave
- Slot
- SlotHold
- Appointment
- Symptom
- AISummary
- Consultation
- Prescription
- Medication
- Notification
- CalendarEvent

The exact table structure will be decided during database design.

34. What the Original Assignment Does NOT Explicitly Require

The following should not be treated as mandatory requirements from the original assignment:

- Advanced Security Architecture

- Audit Trail
- Detailed Status Logging
- LLM Drift Monitoring
- Advanced Hallucination Detection
- AI Model Evaluation Pipeline
- Load Balancer
- API Gateway
- Microservices
- Kubernetes
- S3
- Advanced Monitoring
- Advanced Incident Management
- Multi-region Deployment
- Enterprise Disaster Recovery
- Complex Batch Processing
- Advanced Infrastructure Automation

They may be useful as future enhancements or design considerations, but they are not required to satisfy the original project specification.

35. Important Design Interpretations

Some requirements imply implementation considerations even though the assignment does not prescribe the exact technology.

Double booking

The assignment requires safe handling of simultaneous bookings. Therefore, the implementation will need some form of: Transaction / Concurrency Control / Database Constraint. The exact approach will be selected during development.

Slot hold

The assignment explicitly asks for the slot-hold mechanism to be discussed in the system-design document. Therefore, we need to design one.

Email retry

The assignment explicitly requires background jobs for email retries. Therefore, the email system needs a retry mechanism.

LLM failure

The assignment explicitly requires graceful failure. Therefore, the application must not make successful LLM execution a prerequisite for the entire system functioning.

36. Requirement Priority — MUST HAVE (Original Assignment)

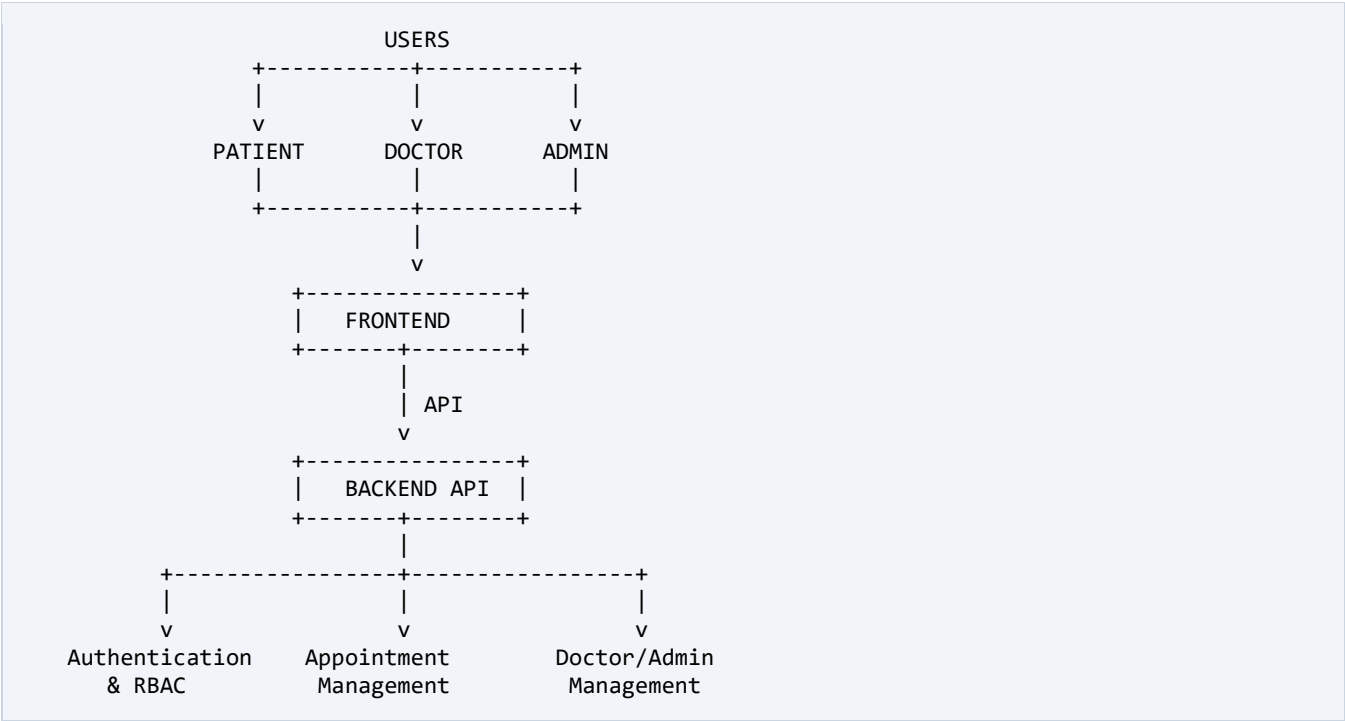
- Patient registration/login
- Doctor login
- Admin functionality
- Role-based authentication
- Doctor profiles
- Specialisation
- Working hours
- Slot duration
- Leave days
- Doctor search
- Appointment booking
- Symptom form
- Double-booking prevention
- Simultaneous booking handling
- Slot hold mechanism
- Doctor leave conflict handling
- Patient notification for affected bookings
- Pre-visit LLM summary
- Urgency level
- Chief complaint
- Three suggested questions
- Doctor post-visit notes
- Prescription
- Post-visit LLM summary
- Medication schedule
- Follow-up steps
- Medication reminders
- Email notifications
- Email retries
- Google Calendar
- OAuth 2.0
- Calendar create/update/delete
- LLM output storage
- Graceful LLM failure
- Backend API
- Frontend
- Database
- Required documentation
- Hosted application

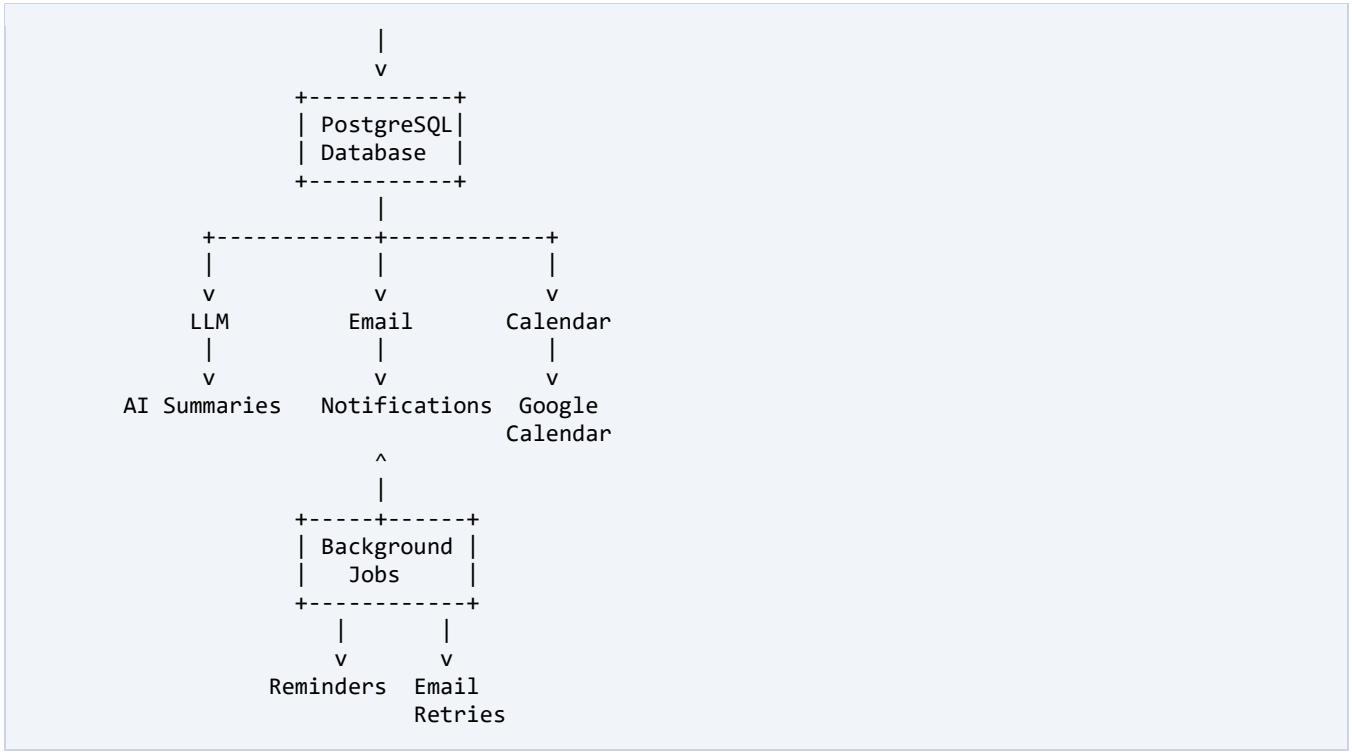
37. Recommended Development Priority

Although all the above are requirements, development should proceed roughly in this order:

- 1. Database
- 2. Authentication + RBAC
- 3. Doctor/Admin Management
- 4. Working Hours + Slot Generation
- 5. Appointment Booking
- 6. Double-Booking + Slot Hold
- 7. Doctor Leave Handling
- 8. Patient Symptoms
- 9. Pre-Visit LLM
- 10. Doctor Consultation + Prescription
- 11. Post-Visit LLM
- 12. Medication Reminders
- 13. Email
- 14. Google Calendar
- 15. Background Jobs + Retry
- 16. Testing
- 17. Deployment
- 18. Documentation

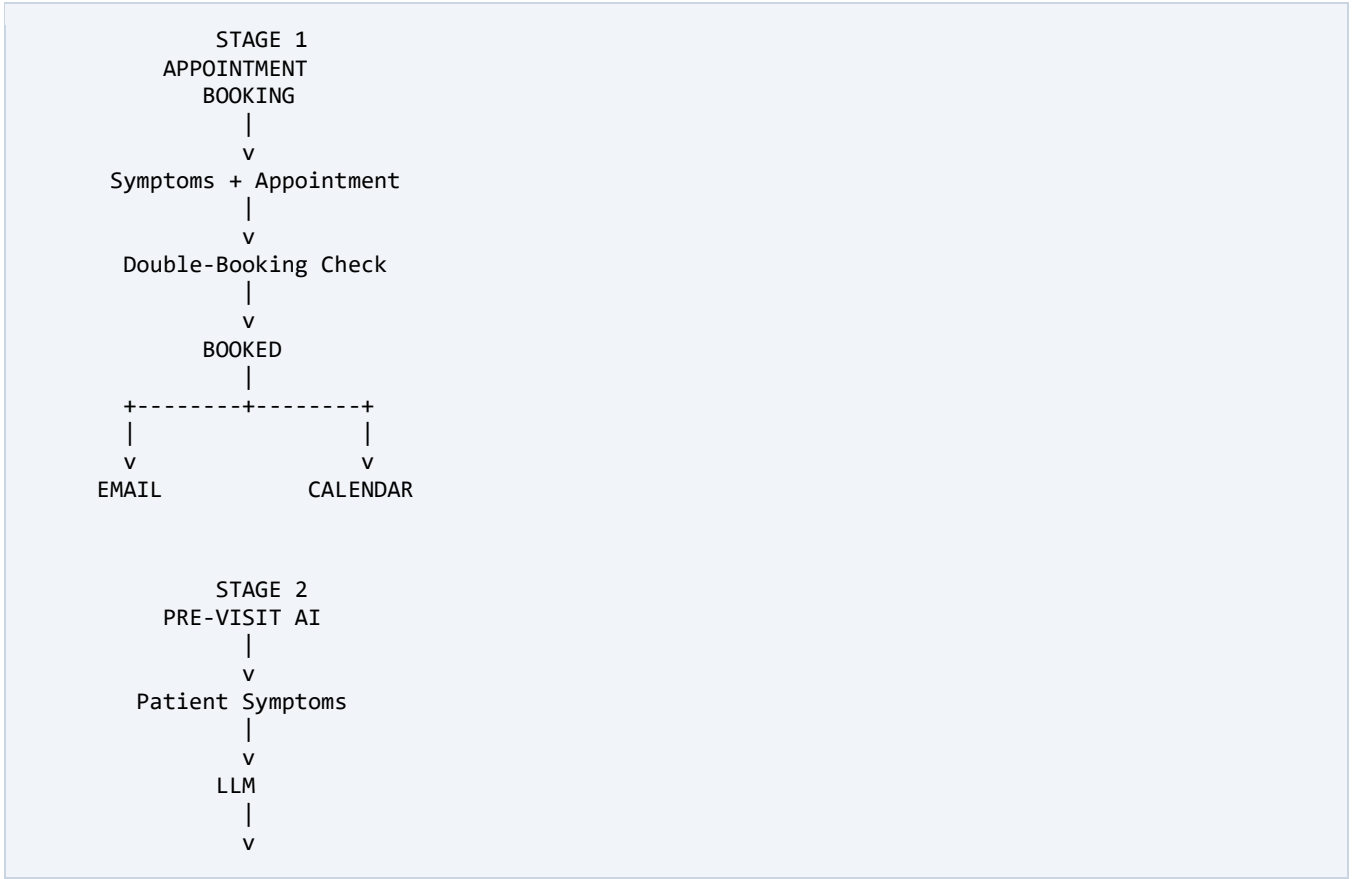
38. Master Architecture Reference

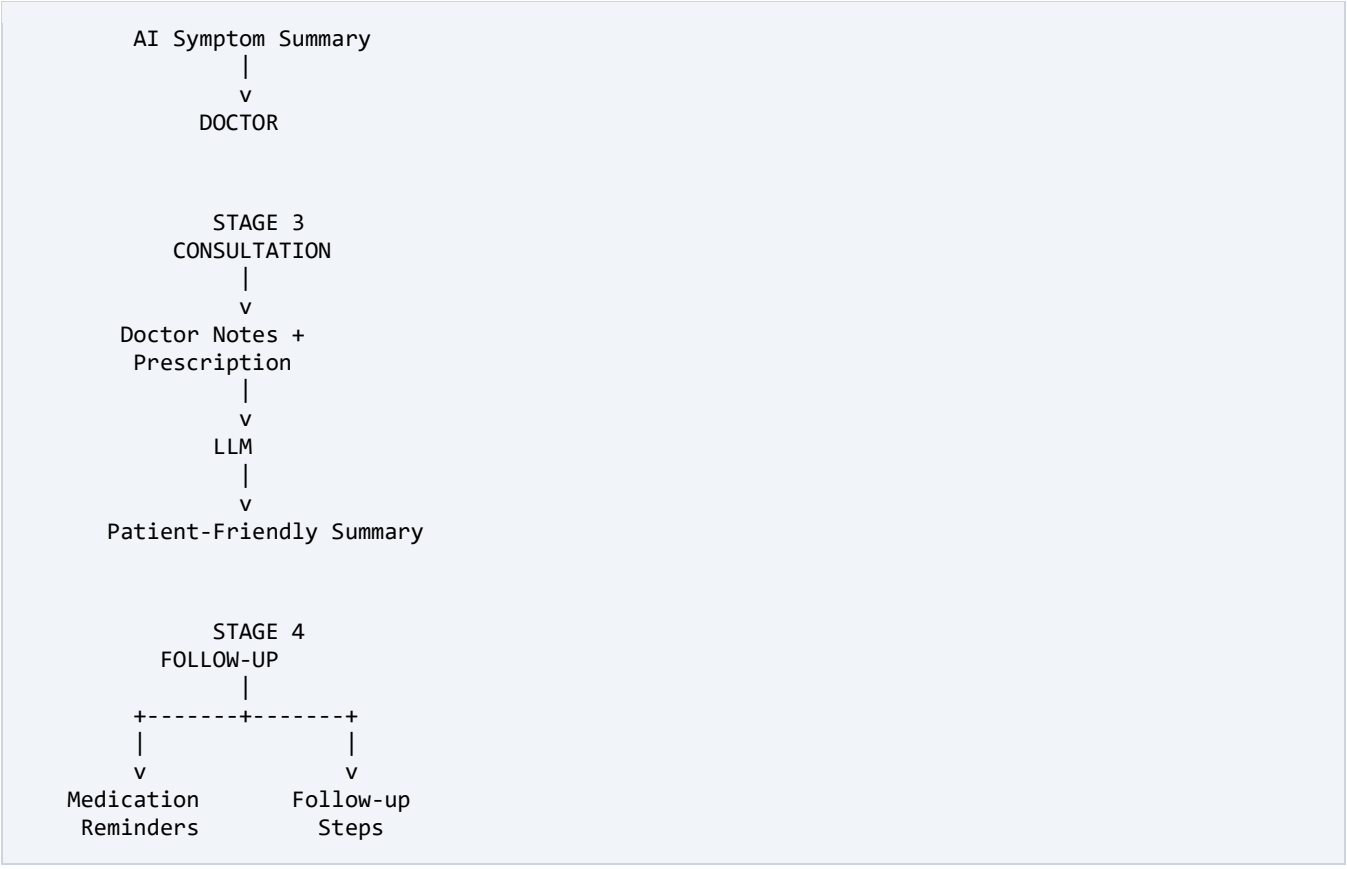




39. Final Project Logic

The entire project can be understood as four major stages:





40. Final Source-of-Truth Checklist

Whenever we review the project in the future, we should ask:

Patient

- ☐ Can patient register?
- ☐ Can patient log in?
- ☐ Can patient search doctors?
- ☐ Can patient search by specialisation?
- ☐ Can patient view available slots?
- ☐ Can patient submit symptoms?
- ☐ Can patient book?
- ☐ Can patient reschedule?
- ☐ Can patient cancel?
- ☐ Can patient receive notifications?
- ☐ Can patient view post-visit information?

Doctor

- ☐ Can doctor log in?
- ☐ Can doctor view appointments?

- ☐ Can doctor view patient symptoms?
- ☐ Can doctor view AI pre-visit summary?
- ☐ Can doctor submit post-visit notes?
- ☐ Can doctor submit prescription?

Admin

- ☐ Can admin create doctors?
- ☐ Can admin manage doctors?
- ☐ Can admin configure specialisation?
- ☐ Can admin configure working hours?
- ☐ Can admin configure slot duration?
- ☐ Can admin manage leave days?

Appointment System

- ☐ Slot generation works
- ☐ Slot hold works
- ☐ Double booking is prevented
- ☐ Simultaneous booking is handled safely
- ☐ Leave conflicts are handled
- ☐ Affected patients are notified

AI

- ☐ Pre-visit summary works
- ☐ Urgency is Low/Medium/High
- ☐ Chief complaint generated
- ☐ Three suggested questions generated
- ☐ Post-visit summary works
- ☐ Medication schedule included
- ☐ Follow-up steps included
- ☐ AI outputs stored in DB
- ☐ LLM failure does not break system

Notifications

- ☐ Booking confirmation
- ☐ Appointment reminder
- ☐ Cancellation notification
- ☐ Leave-conflict notification
- ☐ Email to patient
- ☐ Email to doctor

- ☐ Email retry background job

Calendar

- ☐ OAuth 2.0
- ☐ Create event
- ☐ Update event on reschedule
- ☐ Delete event on cancellation
- ☐ Event for patient
- ☐ Event for doctor

Deliverables

- ☐ Complete source-code ZIP
- ☐ README
- ☐ Setup guide
- ☐ .env.example
- ☐ API documentation
- ☐ DB schema
- ☐ LLM prompts
- ☐ Google Calendar setup
- ☐ Hosted URL
- ☐ 800-word system-design document

41. Final Definition

Healthcare Appointment & Follow-up Manager is a role-based healthcare platform that allows patients to find doctors and book appointments with symptom information, provides doctors with an AI-generated pre-visit symptom summary, enables doctors to record consultation notes and prescriptions, generates a patient-friendly post-visit summary, sends medication and appointment notifications, and synchronizes appointments with Google Calendar while safely handling slot conflicts, doctor leave, background jobs, and LLM failures.

How We Should Use This Document Going Forward

Treat this as the baseline specification, not as the implementation.

```
          |
          v
+----------------+
| Implementation |
+----------------+
          |
          v
+----------------+
| Testing        |
+----------------+
          |
          v
+----------------+
| Deployment     |
+----------------+
```

So, when we later design the database, APIs, folder structure, frontend, backend, LLM integration, slot-locking strategy, email system, Google Calendar integration, or deployment, we should continuously compare our work against this document.

This is the version I recommend you keep as your final project-requirement reference.